Recursion Immersion: Application for Learning Recursion (Development Track)

Julia Nething (Solo Project)
nething@gatech.edu

***Abstract*—**Recursion is a topic that is difficult for novice programmers for learn and difficult for instructors to teach. There are many strategies for teaching recursion but no one-size-fits-all approach that works for the majority of students. This application aims to use research on recursion teaching strategies to combine many of those techniques into an all-in-one platform where students can try many strategies.

1 INTRODUCTION AND BACKGROUND

Recursion is a topic that is typically introduced to computer science students relatively early on but is very difficult for them to conceptualize and learn. Unfortunately, this often leads to students feeling like they are not capable of being successful in computer science simply because they struggle with recursion.

As a teacher of novice adult programmers, this is a common problem that my students face. The typical advice is to "just practice", but it is very difficult for students to practice a concept that they do not understand. My typical strategy is to walk through a recursive problem with them, break it down into as many small steps as possible, and then encourage them to practice it on their own. However, there are many ways to approach recursive problems and not all strategies will resonate with every student. In addition, there are many types of recursive problems so some students may excel at certain types while they consistently struggle with other types.

The goal for this semester-long project was to learn more about the research that exists for how to teach recursion to adults and then create a corresponding application that can combine some of those strategies into an all-in-one platform. By combining many different strategies, students can then try many different techniques until they find the one that makes the most sense to them. They can also hone in on particular problem types that are the most difficult for them to customize their learning plan.

2 RELATED WORK AND EXISTING SOLUTIONS

My research at the beginning of the semester led me to find that professors at the undergraduate level are divided on their strategy for teaching recursion to their students.

2.1 Structural Recursion

A common strategy was to teach structural recursion, which includes any physical structure such as a bulls-eye or Russian nesting dolls (Kruse, 1982). One professor even argued that structural recursion would be less intimidating if it was taught earlier in the curriculum before more common for loops (Bruce, Danyluk, and Murtagh, 2005).

2.2 Mathematical Recursion

Some teachers combined structural recursion with a more mathematical and abstract approach by using geometric shapes or fractals (Elenbogen and O'Kennon, 1988) (Gordon, 2006). Others used a pure mathematical approach by using mathematical induction (Ford, 1984) (Wu, Dale, and Bethel, 1998).

2.3 Games

Another strategy was to teach students logic and patterns via games and hope that students could then make that cognitive leap to writing code on their own (Chaffin et al., 2009). For example, at the University of Texas at Austin, students played a game called Cargo-Bot where they had to reconfigure boxes using only recursive strategies (Lee et al., 2014). Another example of this was a study where student volunteered acted out the Tower of Hanoi problem in front of the class (Benander and Benander, 2008). Students in the audience were then encouraged to write down the recursive patterns that they were seeing as the volunteers acted out the problem.

2.4 Recursive Graphs

Recursive graphs is one of the only strategies that I did not have time to implement in my application, but was a very interesting concept. These graphs allow students to visualize the recursive calls and see the order in which the interpreter calls the code (Hsin, 2008).

2.5 Teaching Recursion in Industry

In non-academic settings such as in industry, recursion is typically self taught using YouTube videos or algorithm-focused websites such as Leetcode, Exercism.io, or InterviewCake. Sometimes individuals will write up an interesting blog post about a recursive problem that they completed on Medium or a similar blog hosting website.

2.6 Methodology and Results

There are several problems with the methodology and results of this research that were common throughout.

One problem is that for many of these studies, the students were not divided up into an experimental group and a control group. Instead, all students tried the new curriculum. Many times this was simply due to a small sample size; if there were not that many students in the class to begin with, it was difficult to divide them into two groups.

Another problem is that the studies were usually not replicated. While it might seem easy to replicate these studies across multiple semesters of the same undergraduate class, due to time constraints this usually was not done. Therefore, it is difficult to say if the results were due to the makeup of that particular group of students or if it can be generalized more widely.

Finally, many of the studies did not use quantitative data. Instead of getting measurable results by testing students on recursive problems, many times the results would be evaluated by asking the students if they felt like they understood recursion better than they did before. This is not a good way to measure the effectiveness because students often feel like they understand more than they do. It is not until one tests them on the subject that their misunderstandings are revealed.

3 SOLUTION

3.1 Overall Application

The solution that was developed over the course of the semester was Recursion Immersion, a web-based application with three main sections.

The first section is the High Level Examples page. This page allows users to

watch videos and consider pre and post test questions that focus on recursion from a high level, not focused on code. Good examples of high level recursion problems are real-world problems such as physical puzzles, traversing through a bulls-eye, traversing a tree diagram, and more. There are currently two examples on this page. One is the Tower of Hanoi problem, which a physical puzzle. The other example is a tree traversal problem which shows both a diagram and some code alongside it.

The second section is the Real Code Examples page. This page allows users to watch videos and consider pre and post test questions that focus on recursion from a code level, usually with problems that are not easy to draw out or diagram. There are currently two examples on this page as well. One is a classic permutation problem of people ordering items at a restaurant. The other example is a binary search tree traversal problem from Leetcode. Both of these examples show useful techniques for breaking down recursive problems, such as solving for the easiest example first and then building up from there.

The third and final section is the Practice Problems page. This page allows users to find example coding problems across the internet so that they can perform the technique of dedicated practice. The problems listed on this page include a link to the problem on the other hosted site, the problem's difficulty, and the various categories that the problem fits into (such as Tree or Linked List).

The screenshots of the application can be found in *Appendix 1: Screenshots of the Application*. The pre and post test questions for the High Level Examples page and the Real Code Examples page can all be found in *Appendix 2: Pre and Post Test Questions*.

3.2 Target Audience and Goals

The target audience for this application is novice adult programmers. However, in theory children or teens who are learning how to code could also use the application. Therefore, the audience could be anyone who wants to learn recursion and do not mind using JavaScript (all of the video examples use JavaScript).

The main goal for this application is to give recursion learners a centralized location to find resources to learn and apply recursion techniques. It is also to bridge the gaps between understanding recursion at a high level, being able to code a recursive problem, and being able to tackle any type of recursive problem.

3.3 Technology Stack

The technologies used for this application are as follows:

- *HTML, CSS, and Material UI* for the front end structure and styling
- *React (JavaScript)* for the front end functionality
- *Webpack and Babel* for compilation and transpilation
- *PostgreSQL* for the database to store the example videos, pre and post test questions, and practice problems
- *Node and Express* for the server to handle API requests from the front end
- *Heroku* for hosting
- *Git and GitHub* for version control

3.4 Benefits of the Tool and How it Differs From Existing Tools

The main benefit of this tool is that it is a "choose your own adventure" experience. Students are able to decide for themselves where they are in their understanding of recursion and tailor their journey accordingly. If they don't understand recursion at all, they would start with the High Level Examples page. If they can do simple problems in recursion but not difficult problems, they might start with the High Level Examples page just to get a more solid foundation, or they might jump to the Real Code Examples page. If they are strong on certain types of recursive problems but weaker on others, they can seek out specific resources for those types of problems because all examples and problems are classified by the problem type.

This is different from previous tools because those tools did not adapt to different students easily. Instead, the makers of those tools or the designers of those classes decided on a couple of techniques and required students to complete tasks in a particular order. This tool is much more flexible and lets the students gravitate towards the examples and problems that they need the most help with.

4 METHODOLOGY AND EVALUATION

4.1 Peer Feedback

Throughout the semester, peers gave feedback on the progress behind building the application. There were four rounds of feedback based on the research behind the tool, one round of feedback on the plan for how to build the tool, and two rounds of feedback based on the progress on development of the tool itself.

4.2 Student Feedback

I have given lectures and conducted one-on-one office hours on recursion several times as a technical mentor to students at a coding bootcamp. The students are all adults with some prior coding experience (at least one month) and each class has approximately 25-30 students. Over the course of the semester, these lectures and individual sessions were altered based on the research that was conducted about teaching recursion and based on student feedback on examples they found helpful or not helpful.

The most recent iteration of the recursion lecture was structured around the three aspects of the Recursion Immersion website: a high level example, a real code example, and a practice problem. The lecture began with a live demonstration of the Tower of Hanoi problem, followed by a live version of the Restaurant Permutation problem, then finally followed by a practice problem demonstration using an array flattening problem.

5 RESULTS

5.1 Peer Feedback

The peer feedback from all rounds were overall positive. Several peers wrote that this was a topic that resonated with them, as they also struggled with recursion, and this was a tool they would actually use. A few others commented that the design was clean and easy to follow.

Some peers gave specific feedback that I later implemented, such as moving the order of the pages (originally Practice Problems was on the far left instead of the far right).

Some useful feedback that I was unable to implement include a suggestion to include a visualization of recursive calls, and a suggestion to add a feature for users to bookmark particular problems.

5.2 Student Feedback

Throughout the most recent recursion lecture with the live demonstrations of the Tower of Hanoi and Restaurant Permutation problems, I received verbal and written feedback from students.

For the Tower of Hanoi problem, some students self-reported that they had trou-

ble following the pattern and understanding how it would translate to code. All but two students reported that they felt they could solve the puzzle on their own. One of the students who was concerned about solving the puzzle on their own said that they felt like they understood the high-level pattern but were worried they would get confused with the details.

For the Restaurant Permutation problem, the approach students were shown was to first solve the problem using regular for loops, and then to manipulate the for loops until every loop was structured in the same way. Once the loops were structured in an identical way, it was easy to copy and paste the code in each loop into a recursive function call. All of the students reported that they could create the regular for loop solution on their own. However, some felt confused at the step where we moved the code into a recursive call.

After the lecture, one student reported that they felt this strategy was too complicated and they wanted to continue to use the strategy that they usually used.

I also sent the example videos from the Recursion Immersion site to three students, all of whom self-reported that they were helpful for their understanding.

6 LIMITATIONS

Some clear limitations of this project include the length of time of the project, the sample size of students, and the feedback process from students.

This project only lasted one semester, so I was only able to give recursion lectures or office hours a handful of times to a limited number of students. If this could be repeated across many groups of students for a longer period of time, I could get more meaningful feedback and data in order to understand trends across a wide variety of students.

In addition, the feedback process was based on subjective information such as whether students felt that they understood, rather than objective feedback like test score improvements. It would be more meaningful if students could be tested before and after using the tool to see if their understanding of recursion improved by using the tool.

7 FUTURE WORK

If there was more time, I would like to add authentication with GitHub so that students can keep track of their progress. This was a part of the site that I worked on throughout the semester, however there was a bug that kept logging the user out and I was unable to fix that bug in time. If that could be fixed, then students could not only keep track of which problems they had accomplished, but they could also fill out the pre and post test questions and submit their answers. They could also mark certain problems as ones they found difficult or would like to go back to later. This would open up more possibilities with evaluation as well, because then the site could aggregate data on which problems were most difficult for students and which example videos were most helpful.

There is always room for improvement in terms of adding more practice questions, example videos, and pre and post test questions. In addition, further development could include adding a discussion forum for students so they can share their own answers to various problems.

Another feature that I would like to add is a way to not only filter by certain problem types, but also have a detail page for each problem type. This would be an all-in-one page where one could see the high level examples, real code examples, and practice problems that are associated with a particular problem type (such as Trees or Linked Lists).

Finally, it would be really interesting to add a visualization such as a recursive graph or code insertion tool so that students could see how the interpreter runs each line of code.

8 CONCLUSION

Learning and teaching recursion is a difficult task. There is not one perfect way to approach the topic and different students will resonate with strategies in different ways. In order to allow students to get the specific help that they need, it makes sense to provide a wide variety of strategies and techniques both at a high level (not involving code at all) and at a code level. Once students feel comfortable with the techniques they have learned, then they can implement dedicated practice by trying out a variety of different problems on the practice page. Students can also choose to focus on particular problem types that they struggle with.

While there is limited data so far on whether or not this tool is effective for students learning recursion, early self-reporting from students is optimistic that it may be helpful. I look forward to showing this to more students in the future, adding more quantitative features, and evaluating more thoroughly the effectiveness of this tool.

9 ACKNOWLEDGMENTS

Thank you very much to Dr. Joyner and to my class mentor Laura Schoenike. Laura gave me very helpful feedback throughout the course and for that I am very grateful! Thank you also to the many peers who gave me specific feedback in peer reviews throughout the course. Finally, I would like to thank my students who sat through several iterations of my recursion lectures and gave me good feedback on what was and was not helpful for them.

10 REFERENCES

[1] Benander, Alan C. and Benander, Barbara A. (Winter 2008). "Student Monks - Teaching Recursion in an IS or CS Programming Course Using the Towers of Hanoi". English. In: *Journal of Information Systems Education* 19.4. Copyright - Copyright EDSIG Winter 2008; Document feature - ; Tables; Diagrams; Graphs; Last updated - 2020-11-17, pp. 455–467. URL: https://go.openathens.net/redirector/gatech.edu?url=https://search-proquest-com.eu1.proxy.openathens.net/scholarly-journals/student-monks-teaching-recursion-is-cs/docview/200157798/se-2?accountid=11107.

[2] Bruce, Kim B, Danyluk, Andrea, and Murtagh, Thomas (2005). "Why structural recursion should be taught before arrays in CS 1". In: *SIGCSE bulletin.* 37.1, pp. 246–250. ISSN: 0097-8418.

[3] Chaffin, Amanda, Doran, Katelyn, Hicks, Drew, and Barnes, Tiffany (2009). "Experimental Evaluation of Teaching Recursion in a Video Game". In: *Proceedings of the 2009 ACM SIGGRAPH Symposium on Video Games*. New York, NY, USA: Association for Computing Machinery, pp. 79–86. ISBN: 9781605585147. URL: https://doi.org/10.1145/1581073.1581086.

[4] Elenbogen, Bruce S. and O'Kennon, Martha R. (Feb. 1988). "Teaching Recursion Using Fractals in Prolog". In: *SIGCSE Bull.* 20.1, pp. 263–266. ISSN: 0097-8418. DOI: 10.1145/52965.53029. URL: https://doi.org/10.1145/52965.53029.

[5] Ford, Gary (Jan. 1984). "An Implementation-Independent Approach to Teaching Recursion". In: *SIGCSE Bull.* 16.1, pp. 213–216. ISSN: 0097-8418. DOI: 10.1145/952980.808655. URL: https://doi.org/10.1145/952980.808655.

[6] Gordon, Aaron (Oct. 2006). "Teaching Recursion Using Recursively-Generated Geometric Designs". In: *J. Comput. Sci. Coll.* 22.1, pp. 124–130. ISSN: 1937-4771.

[7] Hsin, Wen-Jung (Apr. 2008). "Teaching Recursion Using Recursion Graphs". In: 23.4, pp. 217–222. ISSN: 1937-4771.

[8] Kruse, Robert L. (Feb. 1982). "On Teaching Recursion". In: *SIGCSE Bull.* 14.1, pp. 92–96. ISSN: 0097-8418. DOI: 10.1145/953051.801346. URL: https://doi.org/10.1145/953051.801346.

[9] Lee, Elynn, Shan, Victoria, Beth, Bradley, and Lin, Calvin (2014). "A Structured Approach to Teaching Recursion Using Cargo-Bot". In: *Proceedings of the Tenth Annual Conference on International Computing Education Research*. ICER '14. Glasgow, Scotland, United Kingdom: Association for Computing Machinery, pp. 59–66. ISBN: 9781450327558. DOI: 10.1145/2632320.2632356. URL: https://doi.org/10.1145/2632320.2632356.

[10] Wu, Cheng-Chih, Dale, Nell B., and Bethel, Lowell J. (Mar. 1998). "Conceptual Models and Cognitive Learning Styles in Teaching Recursion". In: *SIGCSE Bull.* 30.1, pp. 292–296. ISSN: 0097-8418. DOI: 10.1145/274790.274315. URL: https://doi.org/10.1145/274790.274315.

11 APPENDICES

For some reason the appendices are not formatting properly, please scroll down to find the relevant images and tables.

11.1 Appendix 1: Screenshots of the Application

11.2 Appendix 2: Pre and Post Test Questions

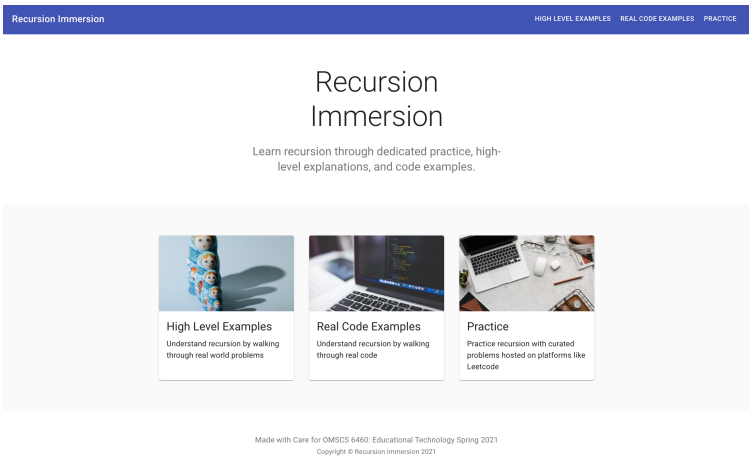

Figure 1—The Splash Page (page that is shown on page load)

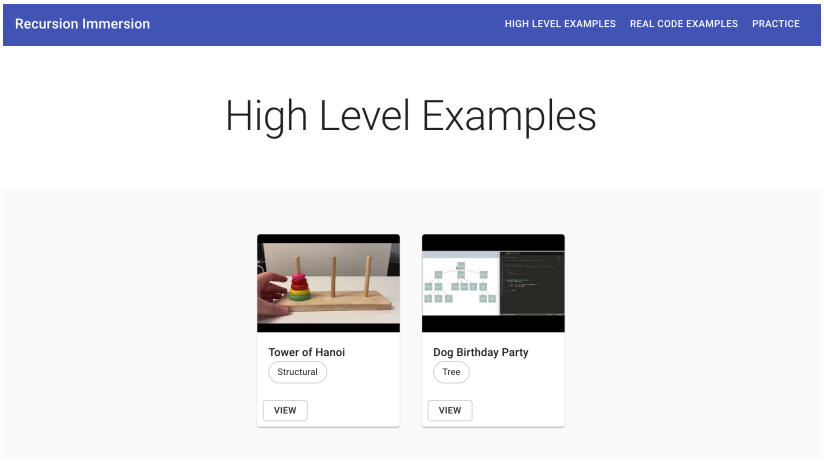

Figure 2—The High Level Examples page

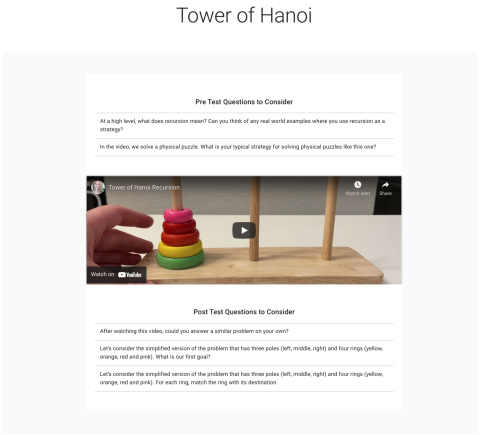

Figure 3—The detail view of one of the high level examples (in this case, the Tower of Hanoi example)

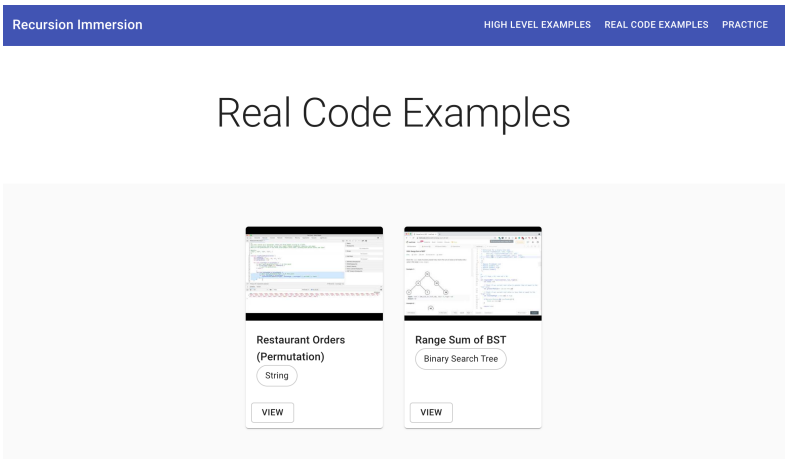

Figure 4—The Real Code Examples page

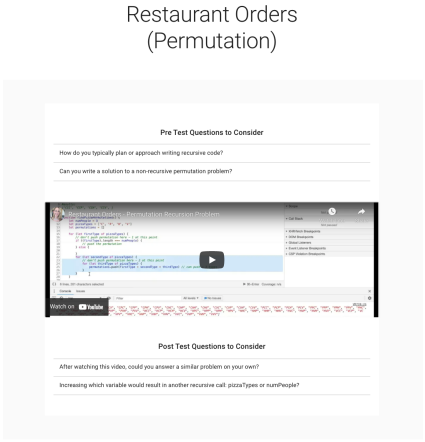

Figure 5—The detail view of one of the real code examples (in this case, the Restaurant Orders Permutation example)

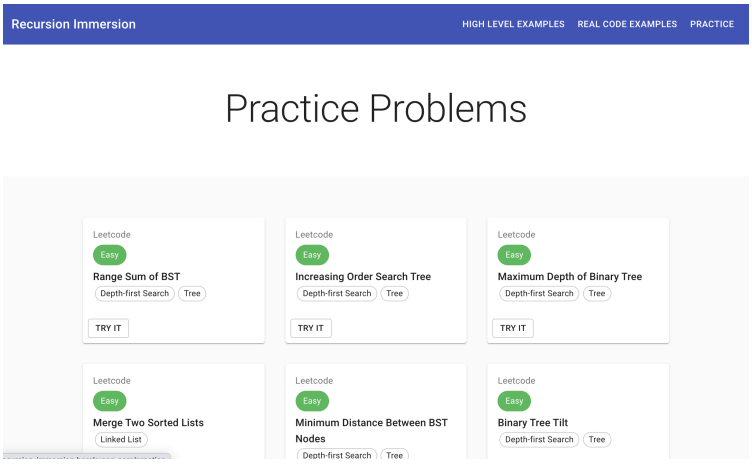

Figure 6—A wide view of the Practice page

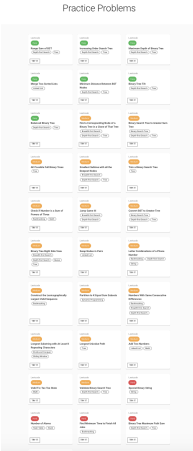

Figure 7—A long view of the Practice page

Table 1—Tower of Hanoi Pre and Post Test Questions

Example	**Test Type**	**Question**
Tower of Hanoi	Pre	At a high level, what does recursion mean? Can you think of any real world examples where you use recursion as a strategy?
Tower of Hanoi	Pre	In the video, we solve a physical puzzle. What is your typical strategy for solving physical puzzles like this one?
Tower of Hanoi	Post	After watching this video, could you answer a similar problem on your own?
Tower of Hanoi	Post	Let's consider the simplified version of the problem that has three poles (left, middle, right) and four rings (yellow, orange, red and pink). What is our first goal?
Tower of Hanoi	Post	Let's consider the simplified version of the problem that has three poles (left, middle, right) and four rings (yellow, orange, red and pink). For each ring, match the ring with its destination.

Table 2—Dog Birthday Party Pre and Post Test Questions

Example	Test Type	Question
Dog Birthday Party	Pre	At a high level, what does recursion mean? Can you think of any real world examples where you use recursion as a strategy?
Dog Birthday Party	Pre	What is a Tree data structure? Can you think of any real world examples where you use Trees to store data?
Dog Birthday Party	Post	After watching this video, could you answer a similar problem on your own?
Dog Birthday Party	Post	Which dogs' totals do we need to know first before we know all of Lu's totals?

Table 3—Restaurant Orders (Permutation) Pre and Post Test Questions

Example	Test Type	Question
Restaurant Orders (Permutation)	Pre	How do you typically plan or approach writing recursive code?
Restaurant Orders (Permutation)	Pre	Can you write a solution to a non-recursive permutation problem?
Restaurant Orders (Permutation)	Post	After watching this video, could you answer a similar problem on your own?
Restaurant Orders (Permutation)	Post	Increasing which variable would result in another recursive call: pizzaTypes or numPeople?

Table 4—Range Sum of BST Pre and Post Test Questions

Example	Test Type	Question
Range Sum of BST	Pre	How do you typically plan or approach writing recursive code?
Range Sum of BST	Pre	What is a Binary Search Tree? Do you know how to traverse a BST in code?
Range Sum of BST	Post	After watching this video, could you answer a similar problem on your own?
Range Sum of BST	Post	If low = 10, high = 20, and root.val = 10, do we need to check the left side of the tree? Why or why not?